RUANG IMAJI / LEARNING GUIDE

Membangun Galeri Seni 3D Interaktif

Panduan langkah demi langkah - Next.js, TypeScript, React Three Fiber, Drei, Three.js, dan GSAP

VERSION 1.2 / Bahasa Indonesia

Versi dokumentasi: 1.2 Kondisi proyek: galeri skylight multi-ruangan, 9 September 2026 Perubahan 1.2: ditambahkan navigasi dua ruangan melalui portal pada lorong kanan. `GalleryCanvas` kini mengelola `roomId`, state machine transisi, overlay fade, dan penguncian input. `galleryRooms.ts` menjadi registry konfigurasi ruangan, `RoomPortal.tsx` menyediakan hitbox 3D dan tombol aksesibel, sedangkan `CameraController` mengabarkan saat kamera tiba sebelum scene ditukar. Antarmuka aplikasi juga telah menggunakan bahasa Inggris.

Daftar Isi

0. Riwayat versi 3
1. Tujuan pembelajaran 4
2. Stack dan peran tiap teknologi 4
3. Menjalankan proyek 5
4. Struktur proyek 5
5. Mental model: DOM dan dunia 3D 5
6. Fundamental koordinat 3D 7
7. Alur arsitektur aplikasi 9
8. Langkah 1 - App Router shell 10
9. Langkah 2 - Client boundary dan Canvas 11
10. Langkah 3 - State selection 11
11. Langkah 4 - Membuat ruangan galeri 12
12. Langkah 5 - Pencahayaan 13
13. Langkah 6 - Data-driven paintings 14
14. Langkah 7 - Komponen Painting 15
15. Langkah 8 - CameraController dan GSAP 16
16. Langkah 9 - Memuat GLB vending machine 17
17. Langkah 10 - Invisible keypad hitbox 17
18. Langkah 11 - Form HTML di dalam scene 19
19. Langkah 12 - CSS overlay 19
20. Alur interaksi lengkap 20
21. Langkah 13 - Multi-room dan portal 20
22. Penjelasan per file 22
23. Masalah umum dan diagnosis 24
24. Performa dan kualitas produksi 25
25. Cara menambah lukisan baru 25
26. Cara membuat tombol vending individual 26
27. Latihan belajar 26
28. Glosarium ringkas 27
29. Ringkasan mental model 28

0. Riwayat versi

Dokumentasi memakai versioning berdasarkan tonggak arsitektur dan pengalaman pengguna, bukan setiap perubahan kecil. Versi terbaru tetap 1.2; tabel berikut menunjukkan evolusi proyek sejak fondasi pertamanya.

Versi	Fokus	Perubahan utama
1.0	Fondasi galeri interaktif	Canvas layar penuh, ruangan persegi panjang, tiga lukisan reusable, selection dan hover, kamera GSAP, popover, model GLB vending machine, invisible keypad hitbox, serta form HTML 3D
1.1	Rekonstruksi ruang skylight	Ruangan dibangun ulang sebagai geometri Three.js berdasarkan referensi interior krem; ditambahkan struktur skylight, konstanta layout bersama, pencahayaan siang, dan penyesuaian posisi objek serta kamera
1.2	Multi-room dan konsistensi UI	Dua ruangan dengan registry bertipe, portal lorong, state machine transisi, fade dan reset kamera, penguncian input, variasi tema/koleksi, serta seluruh wording aplikasi dalam bahasa Inggris

Dari 1.0 ke 1.1

Versi 1.0 membuktikan seluruh interaksi inti: pengguna dapat memilih tiga lukisan pada dinding berbeda, memfokuskan kamera sekitar tiga meter dari objek, kembali ke overview, serta memakai hitbox dan form vending machine. Ruang pada tahap ini masih berupa galeri kotak sederhana.

Versi 1.1 mengubah arsitektur visual tanpa membuang sistem interaksi tersebut:

- memindahkan ukuran ruangan dan kamera home ke `galleryLayout.ts`;
- memisahkan bangunan ke `GalleryRoom.tsx`;
- membangun lantai, dinding, ceruk, balok, ventilasi, dan skylight sebagai mesh 3D;
- mengganti suasana gelap dengan pencahayaan siang dari ambient, hemisphere, directional light, dan spotlight;
- mengatur ulang posisi painting, vending machine, target fokus, dan popover agar cocok dengan ruang baru;
- mempertahankan background sebagai geometri nyata, bukan menempelkan gambar referensi sebagai bidang datar atau mengubahnya otomatis menjadi GLB.

Dari 1.1 ke 1.2

Versi 1.2 memperluas satu ruangan menjadi pengalaman dua ruangan:

- menambahkan `galleryRooms.ts` sebagai registry ID, judul, warna, intensitas cahaya, tujuan portal, dan ketersediaan vending;
- menambahkan `RoomPortal.tsx` dengan invisible hitbox, outline hover, dan tombol HTML yang dapat diakses keyboard;
- menambahkan `roomId` serta fase `idle`, `approaching`, `covering`, dan `revealing` pada `GalleryCanvas`;
- mengurutkan tween kamera, fade gelap, pergantian scene, reset kamera, dan fade pembuka melalui callback GSAP;
- mencegah perpindahan ganda memakai `transitionLock`, prop `disabled`, dan viewport `inert`;
- memasang ulang subtree scene dengan `key={roomId}` agar hover dan form lokal ter-reset;

- membuat Ruang II dengan warna, cahaya, dan susunan artwork berbeda sambil memakai ulang asset yang tersedia;
- mengganti metadata, label, tombol, popover, form, dan pesan aplikasi ke bahasa Inggris.

Perubahan 1.2 tidak menambahkan gerak bebas atau dua bangunan yang tersambung secara fisik. Aplikasi menukar konfigurasi ruangan di dalam satu Canvas ketika overlay tertutup penuh.

1. Tujuan pembelajaran

Dokumen ini membedah proyek `interactive-gallery` yang sedang berjalan, bukan contoh terpisah. Setelah membacanya, Anda diharapkan memahami:

- perbedaan DOM React biasa dan scene graph Three.js;
- fungsi scene, camera, renderer, geometry, material, mesh, texture, dan light;
- cara React Three Fiber menerjemahkan JSX menjadi objek Three.js;
- cara menyusun ruangan 3D dengan sistem koordinat;
- cara membuat komponen lukisan yang reusable dan data-driven;
- cara mengelola selection, hover, pointer event, dan HTML overlay;
- cara menganimasikan kamera dengan GSAP;
- cara memuat model GLB dan menambahkan invisible hitbox;
- cara membangun navigasi multi-room tanpa membuat Canvas baru;
- penyebab z-fighting, posisi hitbox meleset, dan masalah UI 3D umum lainnya;
- arah pengembangan yang aman untuk tahap berikutnya.

Catatan: koordinat dan ukuran di proyek ini adalah unit dunia Three.js. Kita memperlakukannya seperti meter agar mudah bernalar, tetapi Three.js sendiri tidak memaksakan satuan fisik tertentu.

2. Stack dan peran tiap teknologi

Teknologi	Peran di proyek
Next.js App Router	Shell aplikasi, metadata, routing, bundling, dan serving asset publik
React + TypeScript	Komposisi komponen, state UI, props, serta pemeriksaan tipe
Three.js	Mesin 3D: vector, camera, mesh, material, light, texture, dan scene graph
React Three Fiber (R3F)	Renderer React untuk Three.js; menulis scene memakai JSX
Drei	Helper R3F seperti `Html`, `useTexture`, `useGLTF`, `Clone`, dan `Edges`
GSAP	Tween posisi dan arah pandang kamera secara sinematik
CSS	UI DOM: tombol Back, popover, form vending machine, dan aksesibilitas visual

Versi penting saat dokumen dibuat:

TEXT

```
Next.js              16.3.4
React                19.2.8
Three.js             0.185.1
@react-three/fiber   9.7.0
@react-three/drei    10.7.8
GSAP                 3.15.0
```

3. Menjalankan proyek

BASH

```
cd interactive-gallery
yarn install
yarn dev
```

Buka `http://localhost:3000`. Untuk validasi statis:

BASH

```
yarn lint
yarn tsc --noEmit
```

Jika script `tsc` belum ada di `package.json`, jalankan `yarn tsc --noEmit` langsung seperti di atas.

4. Struktur proyek

TEXT

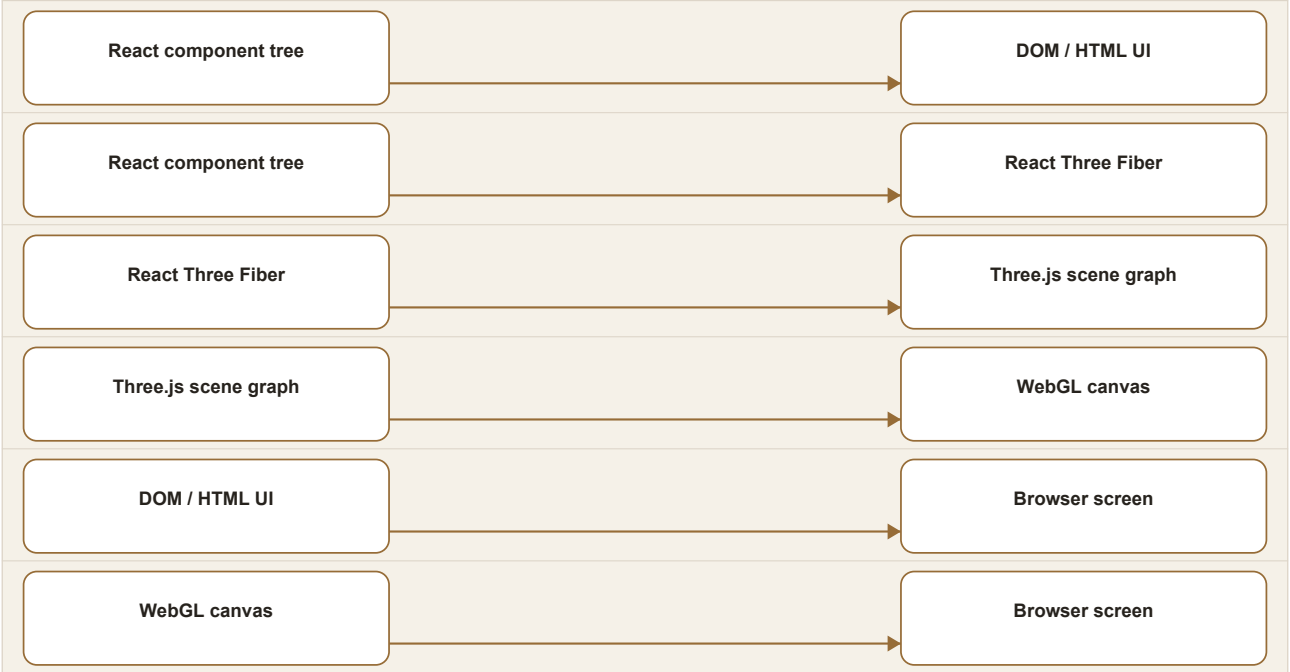
```
interactive-gallery/
├── app/
│   ├── page.tsx                # entry halaman
│   ├── layout.tsx              # root HTML dan metadata
│   └── globals.css             # UI DOM dan layout layar penuh
├── components/gallery/
│   ├── GalleryCanvas.tsx       # client boundary, Canvas, state utama
│   ├── GalleryScene.tsx        # light, data objek, komposisi scene
│   ├── GalleryRoom.tsx         # dinding, skylight, balok, ceruk, ventilasi
│   ├── galleryLayout.ts        # ukuran ruangan dan kamera home bersama
│   ├── galleryRooms.ts         # registry, tema, dan tujuan antar-ruangan
│   ├── RoomPortal.tsx          # portal, hitbox lorong, dan tombol aksesibel
│   ├── Painting.tsx            # lukisan reusable dan popover
│   ├── CameraController.tsx    # animasi kamera GSAP
│   └── VendingMachine.tsx      # GLB, hover, hitbox, dan form
├── public/
│   ├── paintings/              # texture lukisan JPG
│   └── model/                  # soda_vending_machine.glb
├── docs/                       # dokumentasi sumber Markdown
└── output/pdf/                 # dokumentasi PDF final
```

Prinsip modularitasnya: halaman tidak mengetahui detail Three.js, Canvas tidak menggambar ruangan, scene tidak mengurus animasi tween, dan tiap objek interaktif mengelola perilaku lokalnya sendiri.

5. Mental model: DOM dan dunia 3D

Aplikasi ini mempunyai dua lapisan visual:

Diagram hubungan: setiap baris menunjukkan satu arah alur; nama yang sama merujuk node yang sama.



- Tombol Back dan link kredit adalah DOM biasa di atas `<canvas>`.
- Dinding, lampu, lukisan, kamera, dan vending machine adalah objek dalam scene 3D.
- `<Html>` dari Drei menjembatani keduanya: posisinya mengikuti titik 3D, tetapi isi akhirnya adalah DOM sehingga input dan tombol tetap mudah digunakan.

Scene graph bersifat hierarkis:

Diagram hubungan: setiap baris menunjukkan satu arah alur; nama yang sama merujuk node yang sama.

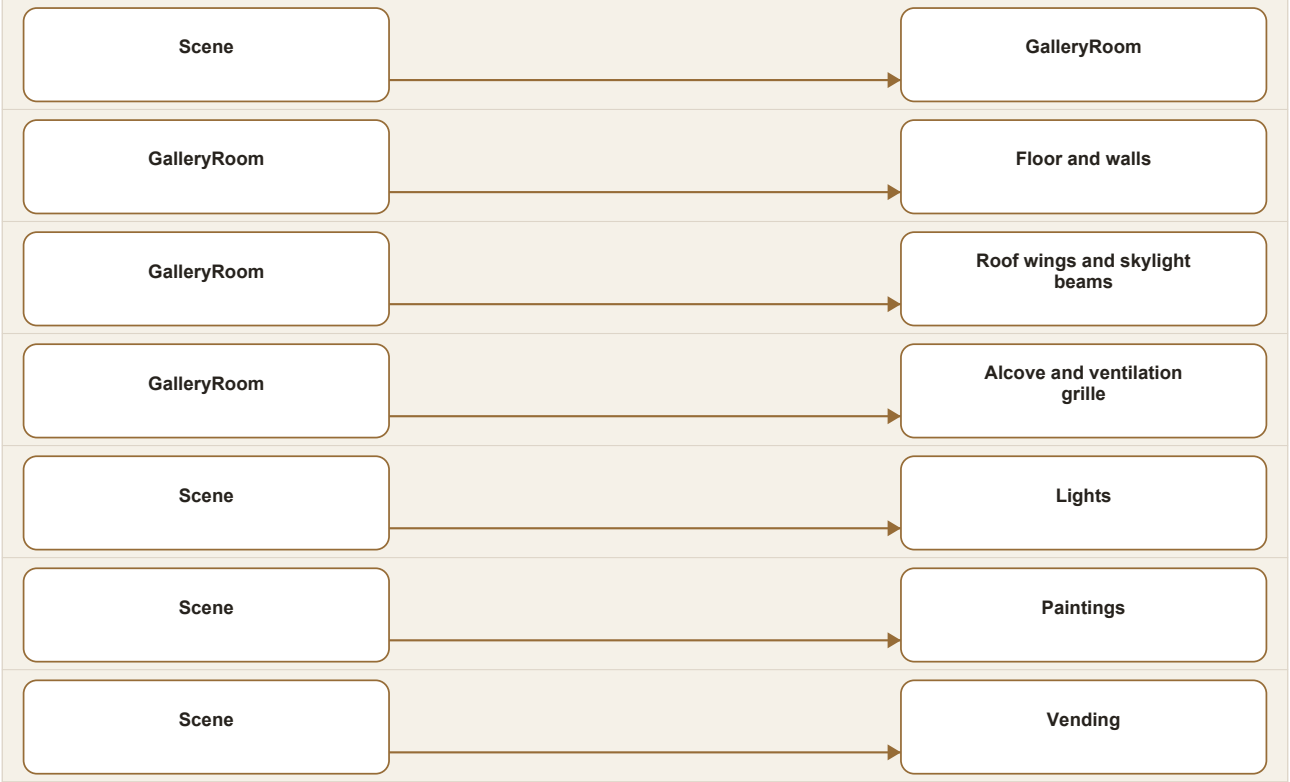
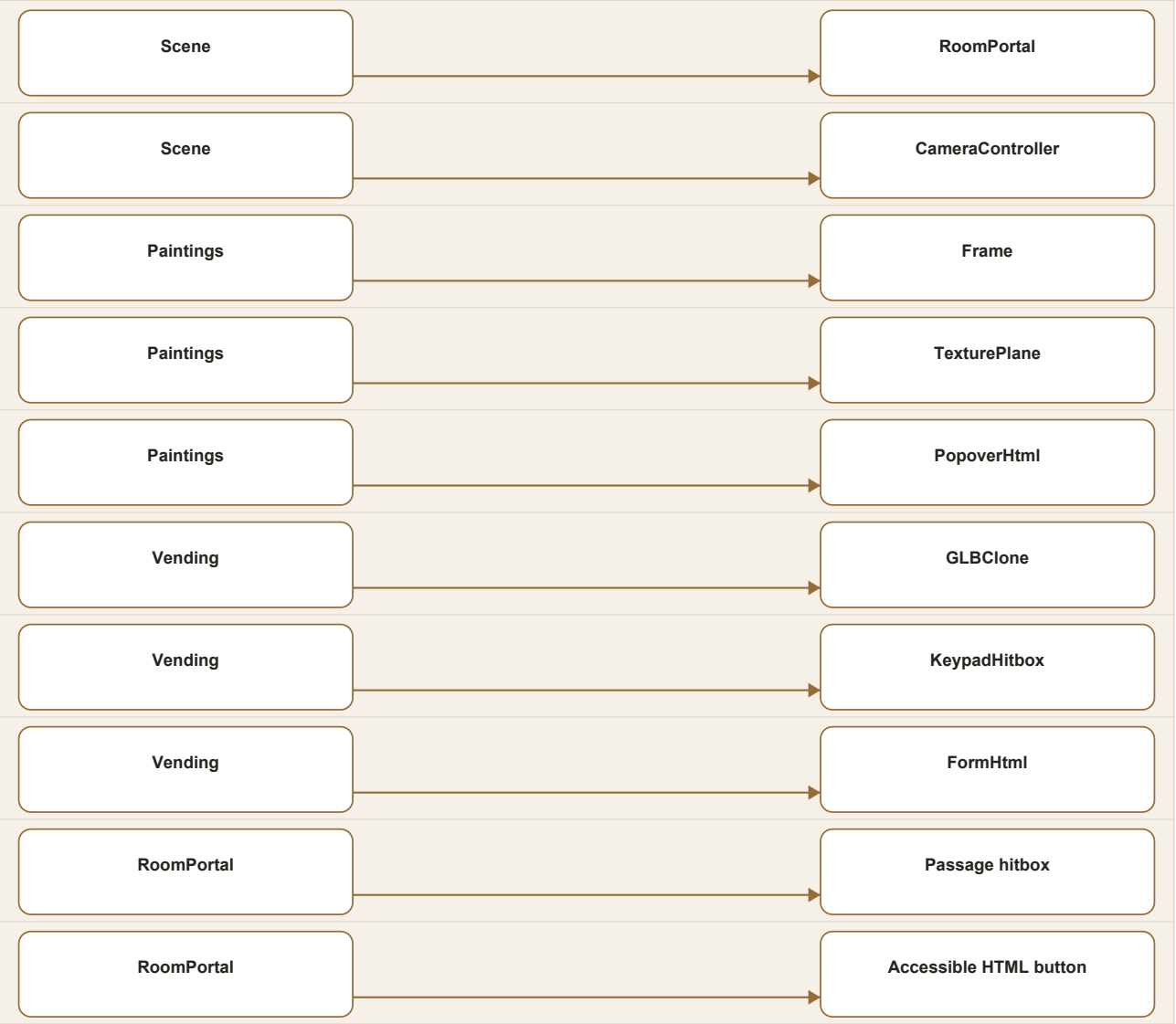


Diagram hubungan: setiap baris menunjukkan satu arah alur; nama yang sama merujuk node yang sama.



Transform parent memengaruhi child. Jika sebuah `<group>` diputar, posisi lokal mesh dan hitbox di dalamnya ikut berputar. Konsep ini penting ketika mengatur lukisan di dinding samping dan hitbox keypad.

6. Fundamental koordinat 3D

Three.js menggunakan sistem koordinat tangan kanan:

TEXT

```
            +Y (atas)
             |
             |
             o------ +X (kanan)
            /
           /
        +Z (ke arah kamera awal)

Dinding belakang berada di Z negatif.
Kamera awal berada di Z positif dan melihat ke Z negatif.
```

Tuple `[x, y, z]` dipakai untuk `position`, sedangkan `rotation` memakai radian `[x, y, z]`.

TSX

```
position={[0, 2.45, -7.86]}
rotation={[0, Math.PI / 2, 0]}
```

- `Math.PI / 2` = 90 derajat.
- `-Math.PI / 2` = -90 derajat.
- Posisi child adalah posisi lokal terhadap parent.
- Setelah parent berotasi, sumbu lokal child tidak selalu searah sumbu dunia.

7. Alur arsitektur aplikasi

Diagram hubungan: setiap baris menunjukkan satu arah alur; nama yang sama merujuk node yang sama.

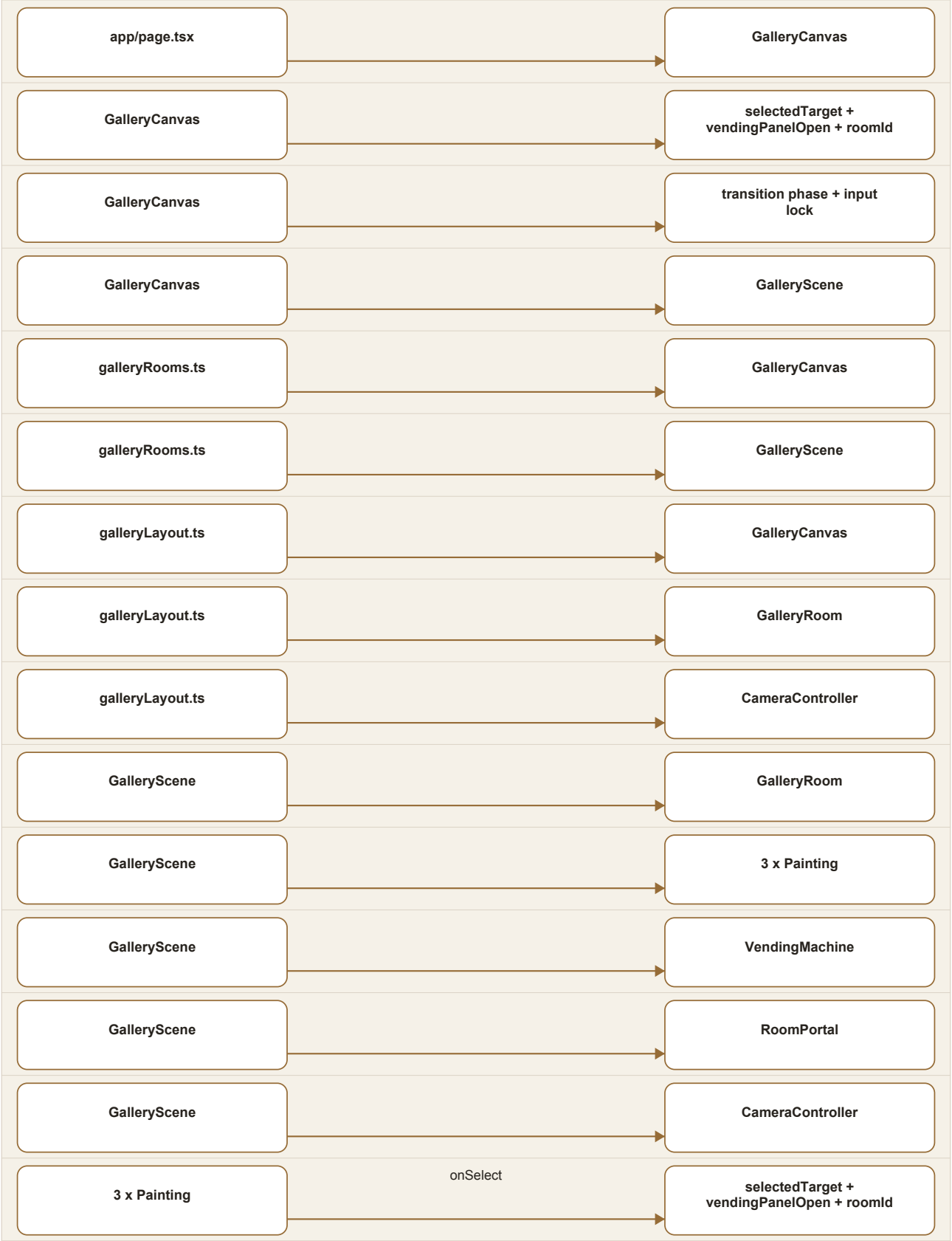
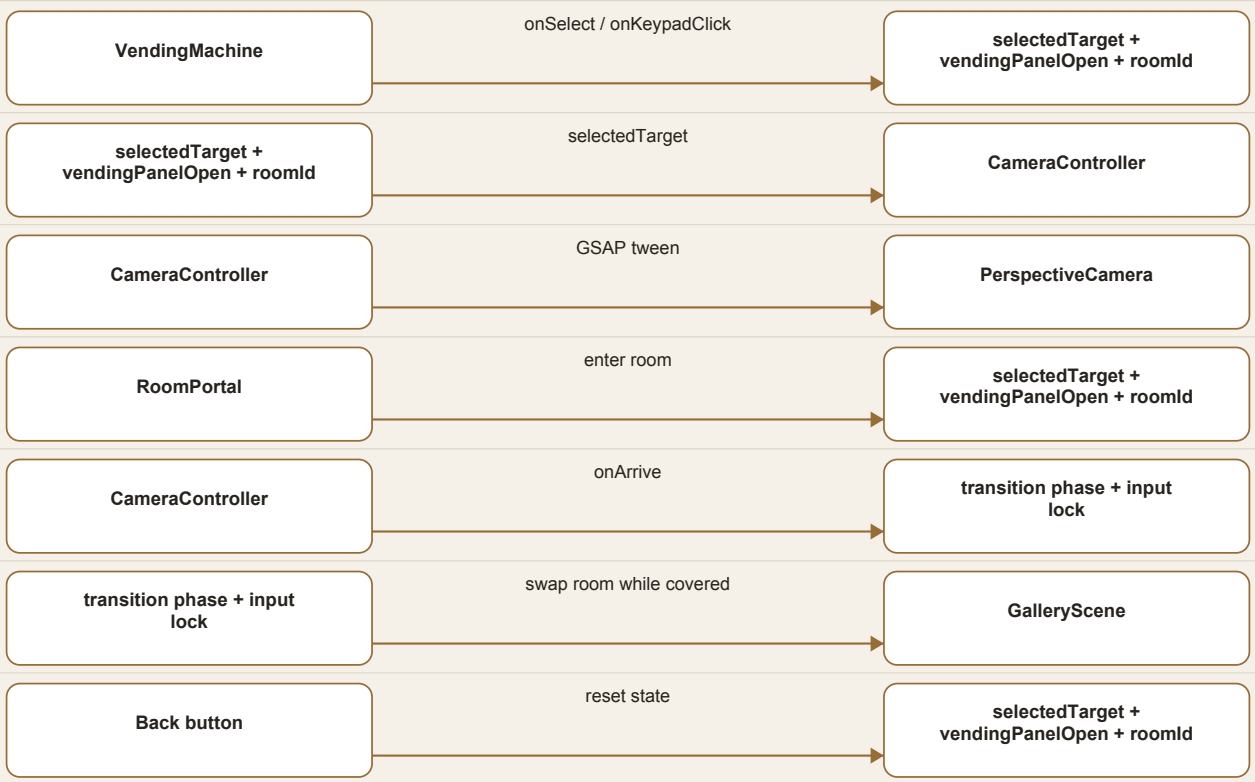


Diagram hubungan: setiap baris menunjukkan satu arah alur; nama yang sama merujuk node yang sama.



Satu sumber kebenaran selection dan ruangan aktif berada di `GalleryCanvas`. Painting, vending machine, dan portal tidak menentukan kamera secara langsung; semuanya hanya mengirim intent atau target. `CameraController` bereaksi terhadap target, lalu callback kedatangan menggerakkan state machine perpindahan ruangan.

8. Langkah 1 - App Router shell

`app/layout.tsx`

File ini membuat root HTML, memuat CSS global, dan mendefinisikan metadata.

TSX

```
export const metadata: Metadata = {
  title: "Imagination Gallery | 3D Art Gallery",
  description: "Explore an interactive 3D art gallery.",
};
```

`<html lang="en">` membantu browser dan pembaca layar mengenali bahasa antarmuka saat ini. `children` adalah halaman aktif yang disisipkan Next.js.

`app/page.tsx`

TSX

```
export default function Home() {
  return (
    <main className="gallery-page" aria-label="Virtual art gallery">
      <GalleryCanvas />
    </main>
  );
}
```

Halaman tetap sederhana. Ia hanya menyediakan landmark `<main>` dan menyerahkan implementasi 3D kepada komponen gallery.

9. Langkah 2 - Client boundary dan Canvas

`GalleryCanvas.tsx` diawali dengan:

TSX

```
"use client";
```

Ini wajib karena komponen memakai state React, event browser, dan WebGL. Komponen server tidak dapat langsung memakai API tersebut.

Konfigurasi Canvas:

TSX

```
<Canvas
  camera={{ position: [...HOME_POSITION], fov: 58, near: 0.1, far: 50 }}
  dpr={[1, 1.75]}
  frameloop="demand"
  gl={{ antialias: true, powerPreference: "high-performance" }}
  shadows="percentage"
>
```

Penjelasan setiap opsi:

- `position`: titik awal PerspectiveCamera.
- `fov`: field of view vertikal; lebih kecil terasa lebih zoom, lebih besar terasa wide-angle.
- `near` dan `far`: jarak clipping camera. Objek di luar rentang tidak dirender.
- `dpr={[1, 1.75]}`: membatasi pixel ratio agar layar retina tidak terlalu mahal.
- `frameloop="demand"`: render hanya saat ada perubahan, hemat GPU untuk scene statis.
- `antialias`: menghaluskan tepi geometri.
- `powerPreference`: meminta GPU performa tinggi bila tersedia.
- `shadows="percentage"`: tipe shadow yang kompatibel dengan versi Three.js saat ini.

`<Suspense fallback={null}>` menunggu texture dan GLB selesai dimuat tanpa merender scene yang belum siap.

10. Langkah 3 - State selection

State pusatnya:

TSX

```
const [selectedTarget, setSelectedTarget] =
  useState<GalleryFocusTarget | null>(null);
const [vendingPanelOpen, setVendingPanelOpen] = useState(false);
const [roomId, setRoomId] = useState<GalleryRoomId>("main");
const [transition, setTransition] = useState<RoomTransition>("idle");
```

`null` berarti camera berada pada tampilan galeri. Sebuah target mempunyai kontrak:

TSX

```
type GalleryFocusTarget = {
  id: string;
  title: string;
  position: readonly [number, number, number];
  cameraTarget: readonly [number, number, number];
};
```

`position` adalah titik yang dipandang camera. `cameraTarget` adalah tujuan posisi camera, bukan arah pandang. Nama ini dapat diperjelas menjadi `cameraPosition` pada refactor mendatang.

Tombol Back muncul secara kondisional ketika ada selection. `handleBack()` mereset selection dan panel vending secara bersamaan, sehingga CameraController otomatis kembali ke home. `roomId` memilih konfigurasi ruangan aktif. `transition` bernilai `idle`, `approaching`, `covering`, atau `revealing`; detailnya dibahas pada langkah multi-room.

11. Langkah 4 - Membuat ruangan galeri

Ukuran dasar ruangan sekarang disimpan dalam `galleryLayout.ts`. `GalleryRoom.tsx` mengimpor `GALLERY_ROOM` dengan alias `ROOM`:

TSX

```
export const GALLERY_ROOM = {
  width: 10,
  height: 5.8,
  depth: 16,
  wallThickness: 0.2,
  skylightWidth: 3.6,
} as const;
```

Helper `RoomBox` membentuk bagian bangunan dari pasangan geometry dan material. Ini cuplikan inti helper tersebut:

TSX

```
<mesh castShadow receiveShadow position={position}>
  <boxGeometry args={size} />
  <meshStandardMaterial color={color} roughness={roughness} />
</mesh>
```

Dalam Three.js:

TEXT

```
Mesh = Geometry + Material + Transform
```

- Geometry mendefinisikan bentuk.
- Material mendefinisikan bagaimana permukaan bereaksi terhadap cahaya.
- Transform adalah position, rotation, dan scale.
- `receiveShadow` membuat permukaan menerima bayangan.

Floor berada sedikit di bawah `y=0`. Back wall berada di `z=-depth/2`. Side wall berada di `x=±width/2`.

Mengapa memakai geometri dan bukan PNG atau GLB?

PNG dapat mengisi layar, tetapi foto perspektif tunggal tidak memiliki kedalaman bangunan. Saat kamera bergerak, foto itu tidak memperlihatkan sisi ruangan baru. Geometri Three.js menyediakan perspektif, bayangan, dan occlusion yang mengikuti kamera. GLB adalah pilihan untuk bangunan kompleks yang dibuat di Blender; untuk bentuk kotak pada referensi ini, komponen JSX sudah cukup. Background scene sebenarnya adalah warna langit `#d5e7f2`, terlihat melalui bukaan.

Membaca `GalleryRoom.tsx` langkah demi langkah

1. `RoomBoxProps` membatasi posisi dan ukuran menjadi tuple tiga angka, dengan warna serta roughness opsional. Default warna adalah `#e4dccb`, roughness `0.86`.
2. `halfWidth=5` dan `halfDepth=8` menentukan pusat dinding. Lantai memakai warna `#cbb89b` dan roughness `0.48`; permukaannya material warna solid, belum memakai texture batu.
3. Dinding kanan utama memiliki panjang 14 dan pusat `z=1`, sehingga menyisakan bukaan pada `z=-8` sampai `-6`. Balok di atas bukaan dan tiga bidang ceruk membentuk area masuk yang benar-benar memiliki kedalaman.
4. `roofWingWidth=(10-3.6)/2=3.2` menentukan lebar setiap sayap atap. Loop `[-1, 1]` mencerminkan sayap atap, balok memanjang, trim skylight, dan skirting pada kedua sisi.
5. Balok memanjang berpusat pada `x=±1.8`, `y=5.55`. Lima balok melintang berada pada `z=[-7.8,-4,0,4,7.8]`. Ruang di antaranya dibiarkan terbuka sebagai skylight; belum ada kaca fisik.
6. Ventilasi di dinding belakang tersusun dari casing, panel gelap, dan 32 bilah tipis melalui `Array.from`. Ini geometri dekoratif, tanpa simulasi udara.
7. Semua bagian berada dalam satu group bernama `Skylight gallery architecture`. Group memberi struktur scene; tiap `RoomBox` menerima dan menghasilkan bayangan.

Ukuran dasar menggunakan konstanta bersama, tetapi dimensi detail ceruk, balok, dan ventilasi masih berupa angka lokal. Mengubah `GALLERY_ROOM` saja belum otomatis menyesuaikan semua detail maupun posisi lukisan; periksa juga `GalleryRoom.tsx` dan data `GalleryScene.tsx`.

12. Langkah 5 - Pencahayaan

Scene menggunakan ambient light, hemisphere light, directional light, dan empat spotlight:

TSX

```
<ambientLight color="#fff5e7" intensity={0.65} />
<hemisphereLight args={["#edf5ff", "#c8b797", 1.6]} />
```

Ambient light memberi penerangan dasar merata agar area gelap tidak menjadi hitam total. Spotlight memberi arah, fokus, dan suasana galeri.

Hemisphere light memberi gradasi warna langit dan tanah berdasarkan orientasi permukaan. Ini pendekatan pencahayaan lingkungan, bukan simulasi pantulan cahaya penuh. Directional light di `[5,12,4]`, warna `#fff0d4`, intensity `3.2`, mengarah ke target default `[0,0,0]` dan memberi kesan matahari melalui skylight. Balok serta atap yang `castShadow` memotong cahaya menjadi pola bayangan.

Shadow matahari memakai map `2048×2048`, batas kamera shadow `-12..12` pada kedua sumbu, near `0.5`, far `40`, bias `-0.0001`, dan normalBias `0.025`. Empat spotlight sekarang memakai intensity `22` dan map `1024×1024`, sebagai cahaya tambahan untuk objek. Target spotlight mengambil `position` langsung dari data objek agar tidak tertinggal ketika objek dipindah.

Spotlight Three.js membutuhkan `Object3D` sebagai target, bukan tuple biasa:

TSX

```
const targetObject = useMemo(() => {
  const object = new Object3D();
  object.position.set(...target);
  return object;
}, [target]);
```

`useMemo` menjaga identitas target tetap stabil antar-render. `<primitive object={targetObject} />` memasukkan objek Three.js mentah ke scene graph R3F.

Properti penting spotlight:

- `intensity`: kekuatan cahaya;
- `distance`: jangkauan maksimal;
- `angle`: lebar cone dalam radian;
- `penumbra`: kelembutan tepi cone;
- `decay`: penurunan energi berdasarkan jarak;
- `shadow-mapSize`: resolusi texture bayangan;
- `shadow-bias` dan `shadow-normalBias`: mengurangi shadow acne.

13. Langkah 6 - Data-driven paintings

Alih-alih menulis tiga komponen berbeda, semua metadata diletakkan dalam array `paintings`.

TSX

```
type GalleryPainting = GalleryFocusTarget & {
  src: string;
  description: string;
  price: number;
  rotation: Vector3Tuple;
  size: readonly [number, number];
};
```

Lalu dirender dengan map:

TSX

```
{paintings.map((painting) => (
  <Painting
    key={painting.id}
    {...painting}
    selected={selectedTarget?.id === painting.id}
    onSelect={() => onSelectTarget(painting)}
  />
))}
```

Keuntungan pola ini:

- menambah lukisan cukup menambah data;
- bentuk dan interaksi tetap konsisten;
- TypeScript memastikan setiap entri lengkap;
- camera target menjadi milik masing-masing objek.

Penempatan saat ini:

Lukisan	Dinding	Rotation Y	Camera sekitar 3 m di depan
Danau Fajar	kiri	`Math.PI / 2`	`[-1.86, 2.45, -2.2]`
Champions 1999	belakang	`0`	`[0, 2.45, -4.86]`
Kota Bulan Sabit	kanan	`-Math.PI / 2`	`[1.86, 2.45, -2.2]`

Pusat lukisan kiri adalah `[-4.86,2.45,-2.2]`, tengah `[0,2.45,-7.86]`, dan kanan `[4.86,2.45,-2.2]`. Vending machine berada di `[-3.3,1.88,-6.25]`, dengan kamera fokus `[-1.18,1.9,-4.13]`. Jarak fokus lukisan tepat 3 unit; vending sekitar 2.998 unit. Vending digeser ke belakang untuk memberi ruang pandang lukisan kiri.

14. Langkah 7 - Komponen Painting

Memuat texture

TSX

```
const texture = useTexture(src);
```

`useTexture` memuat JPG menjadi `THREE.Texture` dan terintegrasi dengan Suspense. Preload di akhir file memulai pemuatan lebih awal.

Frame dan image plane

Frame memakai box tipis; gambar memakai plane di depannya:

TSX

```
<boxGeometry args={[width + 0.12, height + 0.12, 0.04]} />
<planeGeometry args={[width, height]} />
```

Plane diletakkan pada `z=0.035`, sedikit di depan frame. Jarak kecil ini penting. Jika dua permukaan hampir coplanar, depth buffer bingung menentukan mana yang di depan dan muncul kedipan bernama **z-fighting**.

`meshStandardMaterial` bereaksi terhadap light. `roughness` besar menghasilkan pantulan yang lebih lembut. Saat selected, frame memperoleh sedikit emissive agar state fokus terlihat.

Hover yang halus

`useFrame` berjalan pada frame render aktif:

TSX

```
const targetScale = hovered && !selected ? 1.035 : 1;
const nextScale = MathUtils.damp(
  artworkRef.current.scale.x,
  targetScale,
  12,
  delta,
);
```

`MathUtils.damp` membuat transisi frame-rate independent. `delta` adalah waktu sejak frame sebelumnya. Scale hover dinonaktifkan ketika selected agar tidak terjadi zoom tambahan saat camera sudah dekat.

Karena Canvas menggunakan render on demand, `invalidate()` dipanggil selama nilai belum mencapai target.

Pointer event

```tsx
event.stopPropagation();
setHovered(active);
document.body.style.cursor = active ? "pointer" : "default";
```

R3F melakukan raycasting dari pointer ke object 3D. `stopPropagation()` mencegah event melanjutkan ke mesh di belakangnya.

Popover

Ada dua mode:

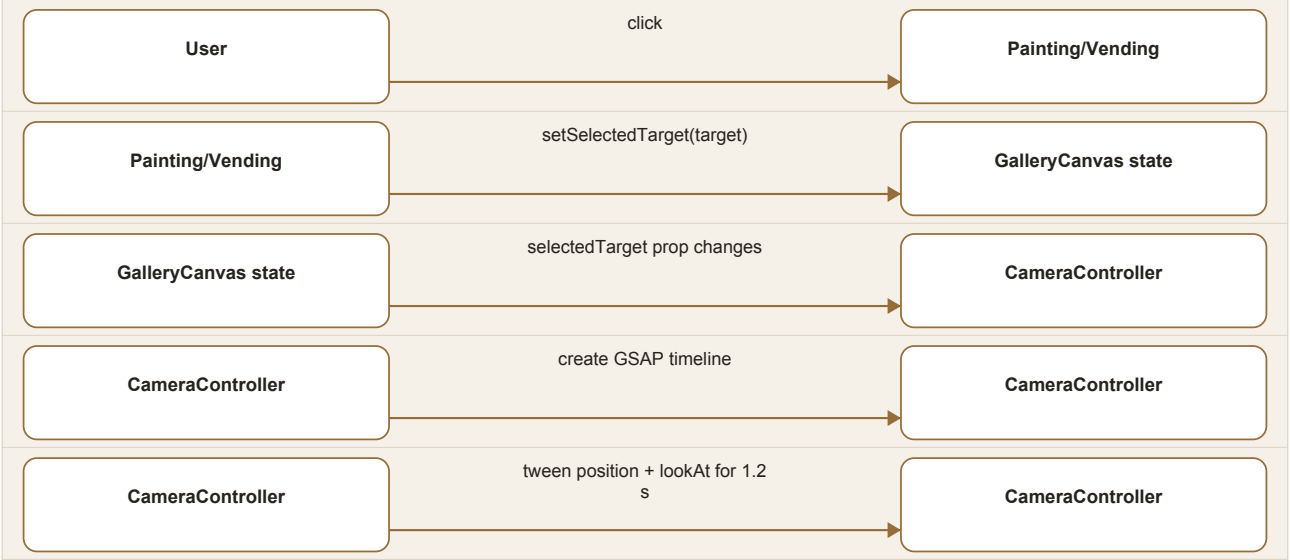
- belum selected: muncul saat hover di atas lukisan;
- selected: selalu muncul di sebelah kiri lukisan.

`<Html>` memproyeksikan anchor 3D ke posisi DOM. `distanceFactor` mengontrol hubungan jarak 3D dengan skala HTML. Popover selected memakai anchor tepi kiri lokal agar panel tidak menutupi gambar.

15. Langkah 8 - CameraController dan GSAP

Alur saat selection berubah:

Diagram hubungan: setiap baris menunjukkan satu arah alur; nama yang sama merujuk node yang sama.



Camera memiliki dua hal yang dianimasikan bersamaan:

1. `camera.position` menuju `cameraTarget` milik object;
2. vector `lookAt` menuju pusat object.

```tsx
timeline
  .to(camera.position, { x, y, z }, 0)
  .to(lookAt, { x, y, z }, 0);
```

Argumen posisi `0` membuat kedua tween mulai pada waktu yang sama. Ease `power2.inOut` mempercepat dengan lembut di awal lalu memperlambat menjelang tujuan.

Pada setiap update:

TSX

```
camera.lookAt(lookAt);
camera.updateProjectionMatrix();
invalidate();
```

Cleanup `timeline.kill()` mencegah tween lama terus berjalan bila pengguna memilih objek lain dengan cepat.

Saat selection `null`, destination berubah menjadi `HOME_POSITION` dan focus menjadi `HOME_LOOK_AT`; mekanisme yang sama membawa camera kembali.

Kedua nilai sekarang berasal dari `galleryLayout.ts`:

TSX

```
export const HOME_POSITION = [0, 2.7, 6.5] as const;
export const HOME_LOOK_AT = [0, 2.65, -7.86] as const;
```

`GalleryCanvas` memakai `HOME_POSITION` yang sama untuk kamera awal. `CameraController` mengimpornya untuk Back, sehingga posisi awal dan posisi kembali tidak memiliki salinan konstanta yang bisa berbeda.

16. Langkah 9 - Memuat GLB vending machine

GLB adalah container biner glTF. Satu file dapat membawa hierarchy mesh, material PBR, texture, camera, animation, dan metadata.

TSX

```
const { scene } = useGLTF(MODEL_PATH);
<Clone object={scene} castShadow receiveShadow />
```

Mengapa memakai `Clone`? Object Three.js sebaiknya tidak dipasang sebagai child di lebih dari satu parent. Clone membuat instance aman jika model kelak digunakan berkali-kali.

TSX

```
<group
  position={[...position]}
  rotation={[0, Math.PI + Math.PI / 5, 0]}
  scale={MODEL_SCALE}
>
```

Group menjadi transform root. Model, label, hitbox, dan form semuanya memakai ruang lokal yang sama. Material dan texture sudah tertanam dalam GLB; lighting scene tetap menentukan seberapa jelas desainnya terlihat.

Model saat ini berasal dari RasenDan di Sketchfab dan menggunakan lisensi CC BY 4.0. Karena itu link kredit ditampilkan permanen di UI.

17. Langkah 10 - Invisible keypad hitbox

Model tidak harus dipecah untuk interaksi sederhana. Kita dapat meletakkan mesh transparan di depan area tombol:

TSX

```
const KEYPAD_HITBOX = {
  position: [-0.35, 0.26, -0.47],
  size: [0.17, 0.27, 0.005],
} as const;
```

TSX

```
<mesh position={[...KEYPAD_HITBOX.position]} onClick={handleKeypadClick}>
  <boxGeometry args={[...KEYPAD_HITBOX.size]} />
  <meshBasicMaterial
    transparent
    opacity={0}
    depthWrite={false}
    colorWrite={false}
  />
  {keypadHovered && <Edges color="#ff3b30" lineWidth={2.5} />}
</mesh>
```

Walau tidak berwarna, geometry tetap berpartisipasi dalam raycast. `colorWrite={false}` mencegah permukaan menulis warna dan `depthWrite={false}` mencegahnya menghalangi object lain di depth buffer.

Hitbox hanya dirender ketika vending machine selected. Ini membuat alurnya dua tahap:

Diagram hubungan: setiap baris menunjukkan satu arah alur; nama yang sama merujuk node yang sama.



`Edges` terlihat hanya pada hover sehingga pengguna memperoleh affordance area klik. `console.info` pada handler membantu debugging nama object, world point, dan UV hasil raycast.

Mengapa posisi hitbox sempat meleset?

Angka hitbox berada dalam koordinat lokal group GLB. Group itu dirotasi dan diskalakan. Akibatnya, “geser kiri” di layar tidak selalu berarti mengurangi nilai X lokal. Cara debugging yang aman:

1. tampilkan sementara material dengan opacity 0.2;
2. gunakan warna terang dan `Edges`;
3. ubah satu sumbu saja dalam langkah kecil;
4. periksa dari camera fokus, bukan hanya camera awal;
5. setelah tepat, kembalikan opacity ke 0.

Jika setiap tombol perlu aksi unik, ada dua pilihan:

- buat beberapa hitbox kecil dengan kode `A`, `1`, `2`, dan seterusnya;
- beri nama mesh tombol di Blender lalu cari node berdasarkan `name` dari hasil `useGLTF`.

Model tidak wajib dipecah menjadi banyak file. Penamaan node yang baik di dalam satu GLB biasanya sudah cukup.

18. Langkah 11 - Form HTML di dalam scene

Form vending machine dirender melalui Drei `<Html>`:

TSX

```
<Html
  center
  position={[0, 0.13, -0.78]}
  distanceFactor={4}
  zIndexRange={[100, 0]}
  style={{ pointerEvents: "auto" }}
>
```

`pointerEvents: "auto"` penting karena popover informasional memakai `none`, tetapi form harus menerima input. Event form dihentikan agar klik tombol tidak diteruskan ke group 3D di belakangnya.

`handleSubmit`:

1. mencegah reload browser;
2. memangkas spasi;
3. mengubah kode menjadi uppercase;
4. menampilkan pesan sukses lokal.

Saat ini submit adalah simulasi UI, belum menghubungi backend atau mengeluarkan produk virtual.

19. Langkah 12 - CSS overlay

`globals.css` menangani tiga kategori:

1. reset dan full-screen layout;
2. UI global seperti Back dan model credit;
3. elemen dari Drei `<Html>` seperti painting popover dan vending form.

Pola visual utama memakai warna gelap semi-transparan:

CSS

```
background: rgb(17 15 13 / 30%);
backdrop-filter: blur(16px);
```

Perbedaannya:

- alpha pada background membuat panel tembus pandang;
- backdrop filter mengaburkan scene di belakang panel;
- box shadow memisahkan panel dari gambar;
- `z-index` mengatur urutan DOM, bukan urutan mesh WebGL.

Media query `prefers-reduced-motion` menonaktifkan animasi dan transisi CSS overlay. Tween kamera GSAP serta scale hover dalam `useFrame` belum mengikuti preferensi ini. Pada ruangan terang, `.model-credit` sekarang memakai background gelap 65%, teks terang 90%, padding, dan radius agar atribusi tetap terbaca.

20. Alur interaksi lengkap

Diagram hubungan: setiap baris menunjukkan satu arah alur; nama yang sama merujuk node yang sama.



21. Langkah 13 - Multi-room dan portal

Versi 1.2 memakai satu `<Canvas>` dan menukar konfigurasi scene di dalamnya. Pendekatan ini menjaga renderer, event system, dan context R3F tetap hidup. Ruang I berisi koleksi utama serta vending machine. Ruang II memakai palet hijau lembut, intensitas matahari berbeda, dan susunan ulang tiga texture yang sama.

Registry ruangan

`galleryRooms.ts` memisahkan identitas dan tema ruangan dari komponen renderer:

TSX

```
export type GalleryRoomId = "main" | "studio";

export const GALLERY_ROOMS = {
  main: {
    title: "Room I · Imagination Gallery",
    nextRoom: "studio",
    skyColor: "#d5e7f2",
    sunlight: 3.2,
    hasVending: true,
  },
  studio: {
    title: "Room II · Quiet Gallery",
    nextRoom: "main",
    wallColor: "#b9c9c0",
    sunlight: 2.2,
    hasVending: false,
  },
} satisfies Record<GalleryRoomId, RoomDefinition>;
```

`satisfies` memeriksa bentuk setiap konfigurasi tanpa menghilangkan tipe literal. `nextRoom` hanya boleh menunjuk `GalleryRoomId` yang valid, sehingga typo tujuan ditemukan TypeScript sebelum browser dijalankan.

Portal sebagai objek interaktif

`RoomPortal.tsx` menempatkan `boxGeometry` tipis pada bukaan dinding kanan. Material transparan tetap dapat terkena raycast. Saat hover, opacity kecil dan `Edges` membuat batas portal terlihat. Portal juga mempunyai tombol Drei `<Html>` agar pengguna keyboard dapat memakai Tab dan Enter.

TSX

```
<mesh
  position={[4.87, 1.75, -7]}
  rotation={[0, -Math.PI / 2, 0]}
  onClick={() => onEnter()}
  onPointerEnter={() => setHovered(true)}
>
  <boxGeometry args={[1.8, 3.4, 0.025]} />
  <meshBasicMaterial transparent opacity={hovered ? 0.16 : 0.025} />
  {hovered && <Edges color="#d8b77a" />}
</mesh>
```

Hitbox sengaja mengikuti bidang lorong, bukan seluruh dinding. `disabled` mematikan handler selama transisi supaya satu klik tidak memulai lebih dari satu perpindahan.

State machine transisi

Perpindahan tidak memakai timer tebakan. Callback `onArrive` dari timeline kamera menjadi sinyal bahwa pendekatan selesai:

Diagram hubungan: setiap baris menunjukkan satu arah alur; nama yang sama merujuk node yang sama.

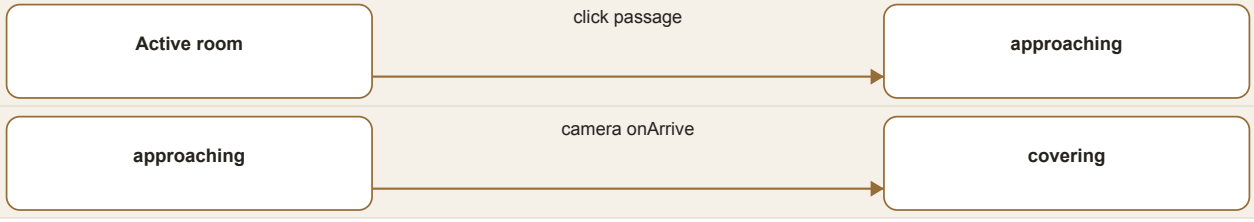
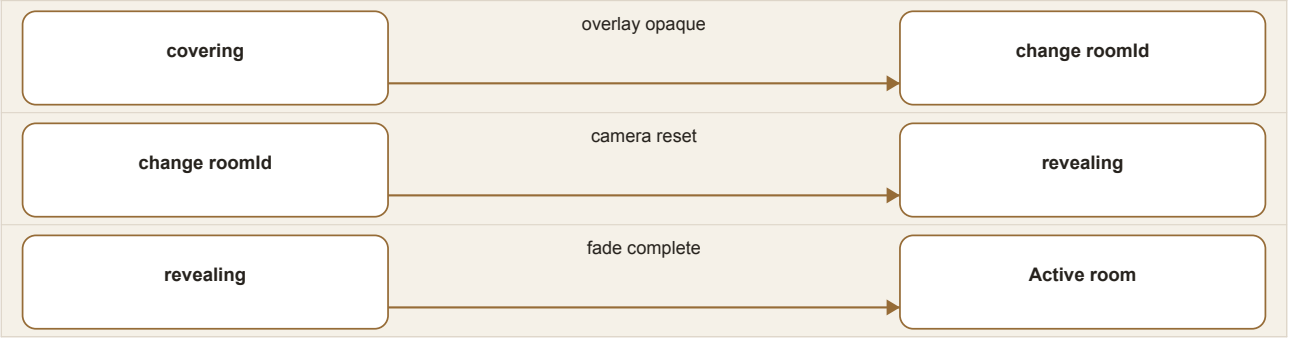


Diagram hubungan: setiap baris menunjukkan satu arah alur; nama yang sama merujuk node yang sama.



Urutan tanggung jawabnya:

1. `handleEnterRoom()` menyetel target kamera ke `PASSAGE_TARGET` dan fase `approaching`.
2. `CameraController` melakukan tween 1,2 detik dan memanggil `onArrive`.
3. `GalleryCanvas` melakukan fade hitam 0,4 detik pada fase `covering`.
4. Setelah overlay penuh, `roomId` diganti, selection serta panel direset, dan fase menjadi `revealing`.
5. `CameraController` memakai `useLayoutEffect` untuk mengembalikan kamera ke `HOME_POSITION` sebelum overlay dibuka selama 0,4 detik.

Scene diberi `key={roomId}`. Ketika ID berubah, React memasang ulang subtree ruangan sehingga state hover dan form lokal tidak bocor ke ruangan berikutnya. `transitionLock` berbasis ref menutup celah sebelum pembaruan state React selesai, sedangkan atribut `inert` pada viewport menonaktifkan input DOM selama transisi. `isolation: isolate` menjaga overlay transisi tetap berada di atas HTML dari Drei.

Implementasi ini adalah pergantian scene, bukan dua bangunan fisik yang dirender bersamaan. Belum ada gerak bebas, collision, persistence setelah reload, maupun kamera home berbeda per ruangan. Batasan ini membuat tahap awal ringan dan mudah diuji.

Menambah ruangan ketiga

1. Tambahkan ID baru ke `GalleryRoomId` dan objeknya ke `GALLERY_ROOMS`.
2. Pastikan `nextRoom` membentuk jalur yang valid.
3. Tambahkan pilihan data lukisan untuk ID itu di `GalleryScene.tsx`.
4. Jika geometri atau kamera home berbeda, pindahkan pilihan tersebut ke konfigurasi ruangan.
5. Uji klik cepat berulang, Back, portal bolak-balik, fokus lukisan, keyboard, dan mobile.

22. Penjelasan per file

`app/page.tsx`

- Entry route `/`.
- Menyediakan semantic `<main>`.
- Tidak menyimpan state atau detail WebGL.

`app/layout.tsx`

- Root document App Router.
- Memuat `globals.css`.
- Menetapkan metadata dan bahasa dokumen.

app/globals.css

- Mengunci aplikasi ke viewport penuh.
- Menata DOM overlay.
- Menyediakan hover/focus state dan reduced motion.
- Tidak mengatur material atau geometry Three.js.

GalleryCanvas.tsx

- Client boundary utama.
- Membuat WebGL Canvas dan camera awal.
- Menjadi owner selection, vending panel, `roomId`, dan fase transisi.
- Mengurutkan approach, fade, room swap, reset, dan reveal.
- Menampilkan tombol Back, label ruangan, overlay, serta atribusi model.

galleryRooms.ts

- Mendefinisikan `GalleryRoomId`, konfigurasi kedua ruangan, dan `PASSAGE_TARGET`.
- Menyimpan judul, subtitle, tujuan, warna, intensitas cahaya, dan ketersediaan vending.
- Menjadikan penambahan ruangan data-driven dan type-safe.

RoomPortal.tsx

- Menambahkan hitbox transparan pada lorong kanan.
- Menampilkan outline ketika hover dan tombol HTML yang aksesibel.
- Mengirim intent perpindahan tanpa mengganti room state sendiri.

galleryLayout.ts

- Mengekspor `GALLERY_ROOM`, `HOME_POSITION`, dan `HOME_LOOK_AT`.
- Dipakai oleh `GalleryRoom`, `GalleryCanvas`, dan `CameraController`.
- Belum menyimpan koordinat artwork atau semua detail bangunan.

GalleryRoom.tsx

- `RoomBoxProps` dan `RoomBox` menyatukan pola box geometry serta material.
- Menyusun lantai, dinding, ceruk kanan, sayap atap, balok skylight, trim, dan ventilasi.
- Loop berpasangan membentuk struktur simetris; loop bilah membentuk ventilasi.
- Tidak memiliki state selection atau event interaksi.

GalleryScene.tsx

- Memilih data painting, vending target, tema, dan lighting berdasarkan ruangan aktif.
- Menyusun semua object 3D.
- Meneruskan callback, bukan menyimpan selection sendiri.
- Menjadi composition root dunia 3D.

Painting.tsx

- Memuat texture.
- Membuat frame dan bidang gambar.
- Mengelola hover lokal.

- Mengirim selection ke parent.
- Merender popover hover dan selected.

`CameraController.tsx`

- Tidak merender geometry (`return null`).
- Mendengarkan perubahan `selectedTarget`.
- Menganimasikan posisi camera dan look-at vector.
- Memanggil `onArrive` dan mereset kamera saat `roomId` berubah.
- Membersihkan timeline lama.

`VendingMachine.tsx`

- Memuat dan clone GLB.
- Mengelola hover model dan hitbox.
- Menampilkan label sederhana.
- Menambahkan invisible hitbox tanpa memodifikasi GLB.
- Menampilkan form DOM yang terikat ke titik 3D.

23. Masalah umum dan diagnosis

Texture hitam atau tidak tampil

- Pastikan path dimulai dari root: `/paintings/painting-1.jpg`.
- File harus berada di `public/paintings`.
- Periksa tab Network untuk status 404.
- Pastikan ada light jika memakai `meshStandardMaterial`.
- Coba `meshBasicMaterial` sementara untuk membedakan masalah texture dan lighting.

Frame berkedip

Penyebab utama adalah z-fighting. Jauhkan texture plane sedikit dari backing atau gunakan geometry frame yang benar-benar memiliki lubang. Jangan menumpuk dua bidang coplanar.

Model GLB tampak gelap/polos

- Pastikan texture benar-benar embedded atau file `.bin` dan image pendamping ikut tersedia.
- Tambah light yang diarahkan ke model.
- Periksa color space texture dan material.
- Jangan mengganti material GLB secara tidak sengaja.

Click tidak masuk ke hitbox

- Pastikan geometry hitbox berada di depan permukaan model.
- Pastikan `selected` benar sehingga hitbox dirender.
- Periksa object lain yang menangkap ray lebih dulu.
- Gunakan `Edges` dan opacity sementara untuk kalibrasi.
- Log `event.object.name` dan `event.point`.

UI `<Html>` terpotong

- Jangan meletakkan anchor terlalu dekat tepi viewport.

- Gunakan posisi berbeda antara mode overview dan focus.
- Sesuaikan `distanceFactor`.
- Untuk UI penting yang harus selalu terlihat, pertimbangkan overlay DOM fixed di luar Canvas.

Dev server terus reconnect

- Hindari menjalankan `next build` pada folder `.next` yang sama saat `next dev` aktif.
- Hentikan proses lama dan jalankan ulang `yarn dev`.
- Periksa error kompilasi pertama di terminal, bukan hanya pesan reconnect browser.

Transisi ruangan macet atau berjalan dua kali

- Pastikan callback `onArrive` hanya dipanggil oleh timeline aktif.
- Kill timeline GSAP pada cleanup.
- Jangan menghapus `transitionLock`; state React saja masih menyisakan celah klik pada frame yang sama.
- Pastikan overlay menyelesaikan fase `covering` dan `revealing` melalui callback completion.
- Periksa bahwa `nextRoom` menunjuk ID yang ada di `GALLERY_ROOMS`.

24. Performa dan kualitas produksi

Yang sudah diterapkan:

- `frameloop="demand"` untuk scene dominan statis;
- batas DPR untuk mengendalikan biaya pixel;
- preload texture dan GLB;
- cleanup GSAP timeline;
- satu Canvas untuk seluruh ruangan dan scene lama dilepas saat room swap;
- input lock serta viewport inert selama transisi;
- komponen dan data terpisah;
- reduced motion pada animasi CSS overlay;
- type-safe tuple dan target contract.

Peningkatan berikutnya:

- kompres texture ke WebP/AVIF atau KTX2;
- kompres geometry GLB dengan Draco/Meshopt;
- gunakan environment map yang terkontrol;
- tambahkan loading progress dan error boundary;
- uji keyboard navigation dan mobile layout;
- pisahkan data artwork ke JSON atau CMS;
- sediakan fallback bila WebGL tidak tersedia;
- audit lisensi seluruh asset.

25. Cara menambah lukisan baru

1. Simpan image di `public/paintings`.
2. Tambahkan item ke array `paintings`.
3. Tentukan `position` dan `rotation` berdasarkan dinding.
4. Tentukan posisi camera sekitar 2-3 unit di depan bidang.

5. Tambahkan preload bila diperlukan.
6. Uji overview, hover, focus, popover, dan Back.

Contoh:

TSX

```
{
  id: "new-artwork",
  src: "/paintings/new-artwork.jpg",
  title: "Judul Baru",
  description: "Deskripsi singkat.",
  price: 12_000_000,
  position: [0, 2.45, -7.86],
  rotation: [0, 0, 0],
  size: [1.8, 2.4],
  cameraTarget: [0, 2.45, -4.86],
}
```

26. Cara membuat tombol vending individual

Untuk input langsung dari keypad, ubah satu hitbox besar menjadi grid hitbox kecil:

TSX

```
const KEYS = [
  { label: "A", position: [-0.04, 0.10, 0] },
  { label: "1", position: [0.00, 0.10, 0] },
  { label: "2", position: [0.04, 0.10, 0] },
];
```

Render setiap key relatif terhadap sebuah `<group position={KEYPAD_ORIGIN}>`. Klik menambahkan karakter ke state `code`. Keuntungan origin group adalah seluruh keypad dapat digeser tanpa menghitung ulang setiap tombol.

Untuk presisi lebih tinggi:

1. buka GLB di Blender;
2. beri nama mesh tombol, misalnya `Key_A`, `Key_1`;
3. export sebagai satu GLB;
4. akses node hasil `useGLTF` berdasarkan nama;
5. pasang handler atau hitbox pada node terkait.

27. Latihan belajar

1. Tambahkan kaca skylight tanpa menutup sudut pandang camera; uji transparansi dan bayangan.
2. Tambahkan satu sculpture GLB di tengah ruangan.
3. Buat highlight spotlight berubah ketika selection berubah.
4. Buat keypad mengisi input tanpa keyboard.
5. Tambahkan validasi kode berdasarkan daftar produk.
6. Buat camera path dua tahap: mendekat lalu framing.
7. Tambahkan loading screen dengan `useProgress` dari Drei.
8. Pindahkan metadata lukisan ke file JSON.
9. Tambahkan Ruang III dengan tema, koleksi, dan arah portal sendiri.

28. Glosarium ringkas

Istilah	Arti
Scene	Root container dunia 3D
Scene graph	Hierarki parent-child seluruh object 3D
Camera	Sudut pandang yang memproyeksikan dunia ke layar
Renderer	Sistem yang menggambar scene dari camera ke canvas
Geometry	Bentuk dan vertex sebuah object
Material	Cara permukaan merespons cahaya dan warna
Mesh	Gabungan geometry dan material
Texture	Image/data yang dipetakan ke permukaan
UV	Koordinat 2D pada permukaan geometry untuk texture
Raycasting	Menembakkan ray dari pointer untuk mencari object yang terkena
GLB/glTF	Format pertukaran scene dan asset 3D
PBR	Model material berbasis sifat fisik
Z-fighting	Kedipan akibat dua permukaan berebut depth yang sama
Local space	Koordinat relatif terhadap parent
World space	Koordinat final terhadap root scene
Tween	Interpolasi nilai dari awal ke tujuan selama durasi tertentu
Invalidate	Meminta R3F merender frame baru pada demand mode
State machine	Model fase eksplisit dan transisi yang diizinkan
Inert	Status DOM yang menonaktifkan focus dan interaksi subtree

29. Ringkasan mental model

Jika Anda memahami alur berikut, Anda sudah memegang fondasi utama proyek ini. Tahap berikutnya bukan sekadar menambah fitur, tetapi memperkuat data model, accessibility, asset pipeline, dan pengujian interaksi.

TEXT

```
Data object menentukan posisi dan camera destination.
        ↓
GalleryScene membangun scene graph.
        ↓
R3F menerjemahkan JSX menjadi object Three.js.
        ↓
Pointer diraycast ke mesh dan hitbox.
        ↓
Selection disimpan di GalleryCanvas.
        ↓
CameraController tween camera dengan GSAP.
        ↓
Portal arrival memicu fade dan pergantian roomId.
        ↓
Scene ruangan baru dipasang dan camera direset.
        ↓
Drei Html menempelkan UI DOM ke titik 3D.
```